

소프트웨어공학

객체지향 방법론 2

- 1 객체지향 기본원리 (캡슐화, 정보은닉, 상속성, 추상화, 객체의 다형성)
- 2 객체지향 언어

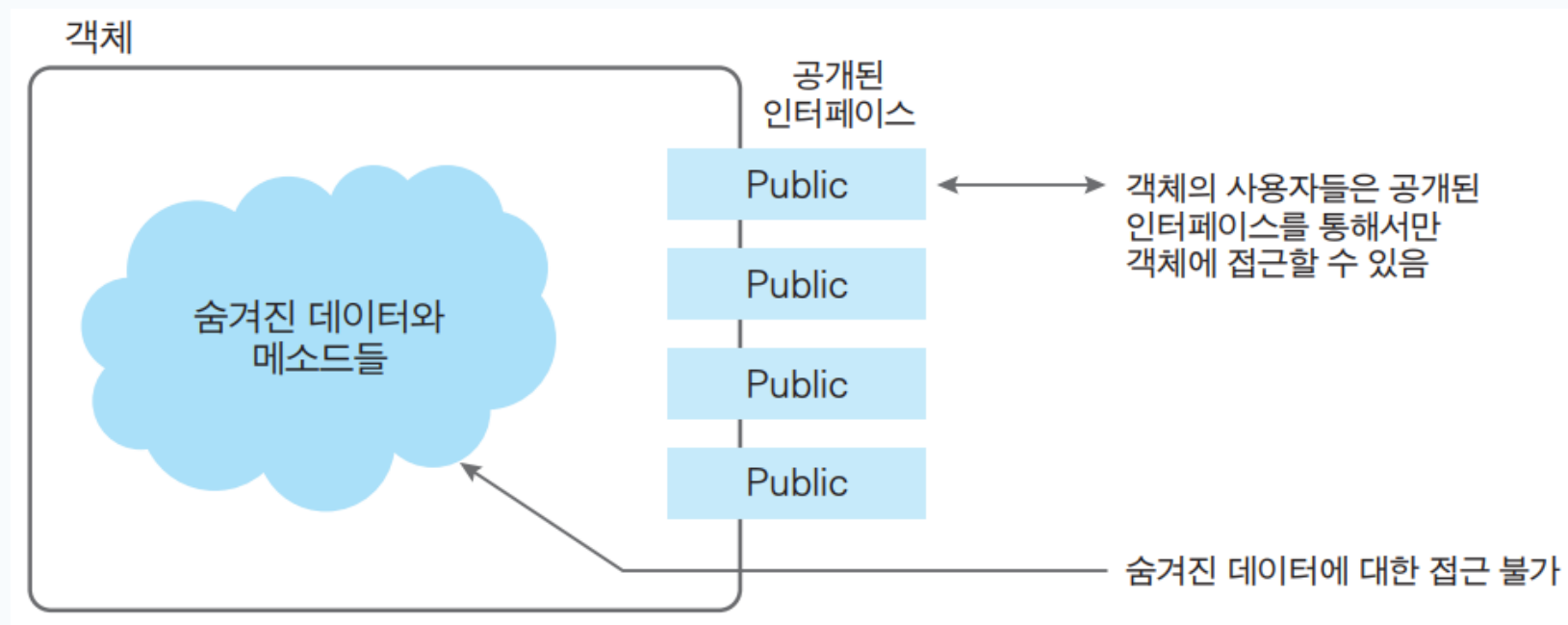
01

객체지향 기본원리 (캡슐화, 정보은닉, 상속성, 추상화, 객체의 다형성)

- 추상화

추상화 (Abstraction)

- 사전적 의미 → “사물이나 표상을 어떤 성질, 공통성, 본질에 착안하여 그것을 추출하여 파악하는 것”이라 정의한다.
- 즉, 객체에서 공통된 속성과 행위를 추출 하는 것이다.
- 추상화는 불필요한 정보는 숨기고 중요한 정보만을 표현함으로써 프로그램을 간단하게 만드는 것이다.
- 공통의 속성과 행위를 찾아서 타입을 정의하는 과정이다.
- 자료 추상화를 통해 정의된 자료형을 추상 자료형이다.
- 추상 자료형은 자료형의 자료 표현과 자료형의 연산을 캡슐화한 것으로 접근 제어를 통해서 자료형의 정보를 은닉할 수 있다.



01

객체지향 기본원리 (캡슐화, 정보은닉, 상속성, 추상화, 객체의 다형성)

- 추상화의 예

- 추상(Abstract)클래스와 인터페이스를 통해 가능.

```
abstract class Car{
    ...
    abstract boolean move(int z);
}

class Sedan extends Car{
    ...
    @Override
    boolean move(int z){
        return z;
    }
}
```

```
interface Moveable{
    void move(int s);
}

class Truck implements Moveable{
    private int speed;

    @Override
    public void move(int s) {
        System.out.println("Truck is move.");
        this.speed = s;
    }
}
```

01

객체지향 기본원리 (캡슐화, 정보은닉, 상속성, 추상화, 객체의 다형성)

- 캡슐화

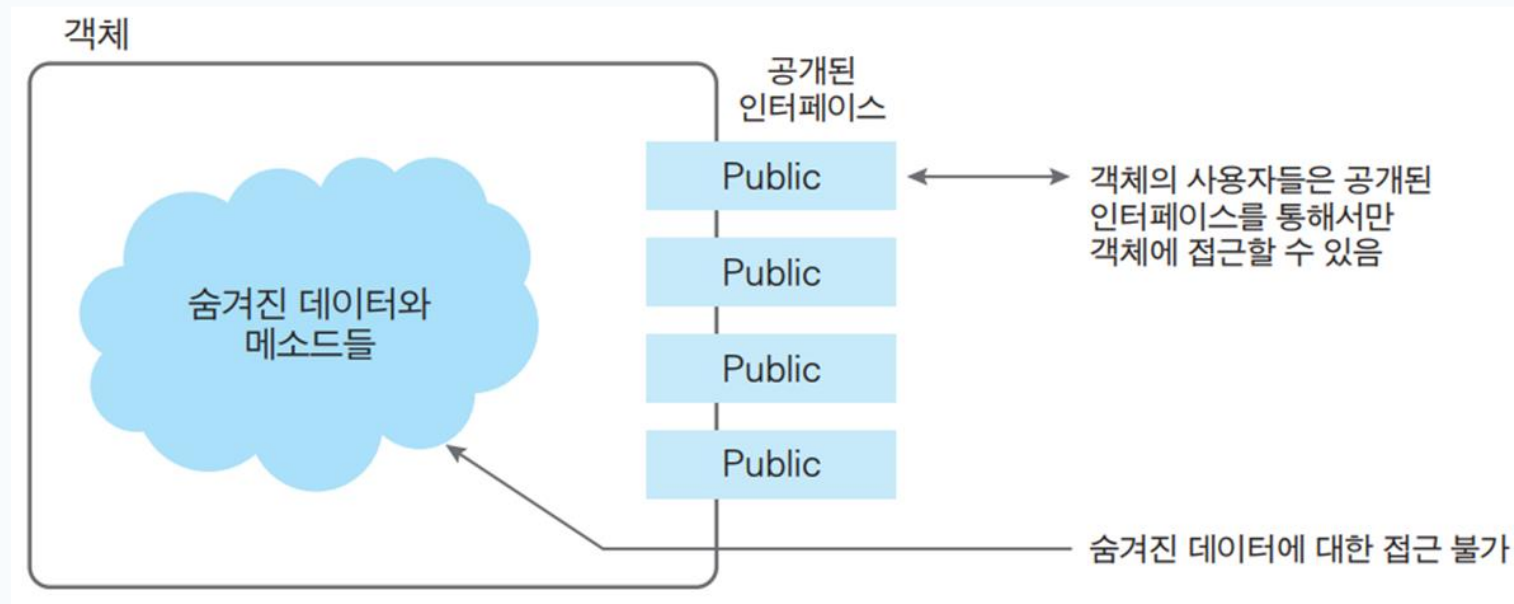
캡슐화 (encapsulation)

- 서로 연관있는 속성(데이터)과 기능(함수)들을 하나의 캡슐(capsule)로 만들어 데이터를 외부로부터 보호하기 위해 데이터와 데이터를 처리하는 함수를 하나로 묶는 것이다.
- 캡슐화를 하는 이유
 - ➔ 데이터 보호(data protection) - 외부로부터 클래스에 정의된 속성과 기능들을 보호
 - ➔ 데이터 은닉(data hiding) - 내부의 동작을 감추고 외부에는 필요한 부분만 노출
- 변경에 대한 오류의 파급 효과가 적다.
- 캡슐화된 객체들은 재사용이 용이하다.
- 인터페이스가 단순해지고 객체 간의 결합도가 낮아진다.

01

객체지향 기본원리 (캡슐화, 정보은닉, 상속성, 추상화, 객체의 다형성)

- 캡슐화의 예



```
public class Person {  
  
    private int age;  
  
    public int getAge() {  
        return age;  
    }  
  
    public void setAge(int age) {  
        if (age >= 0) {  
            this.age = age;  
        }  
    }  
}
```

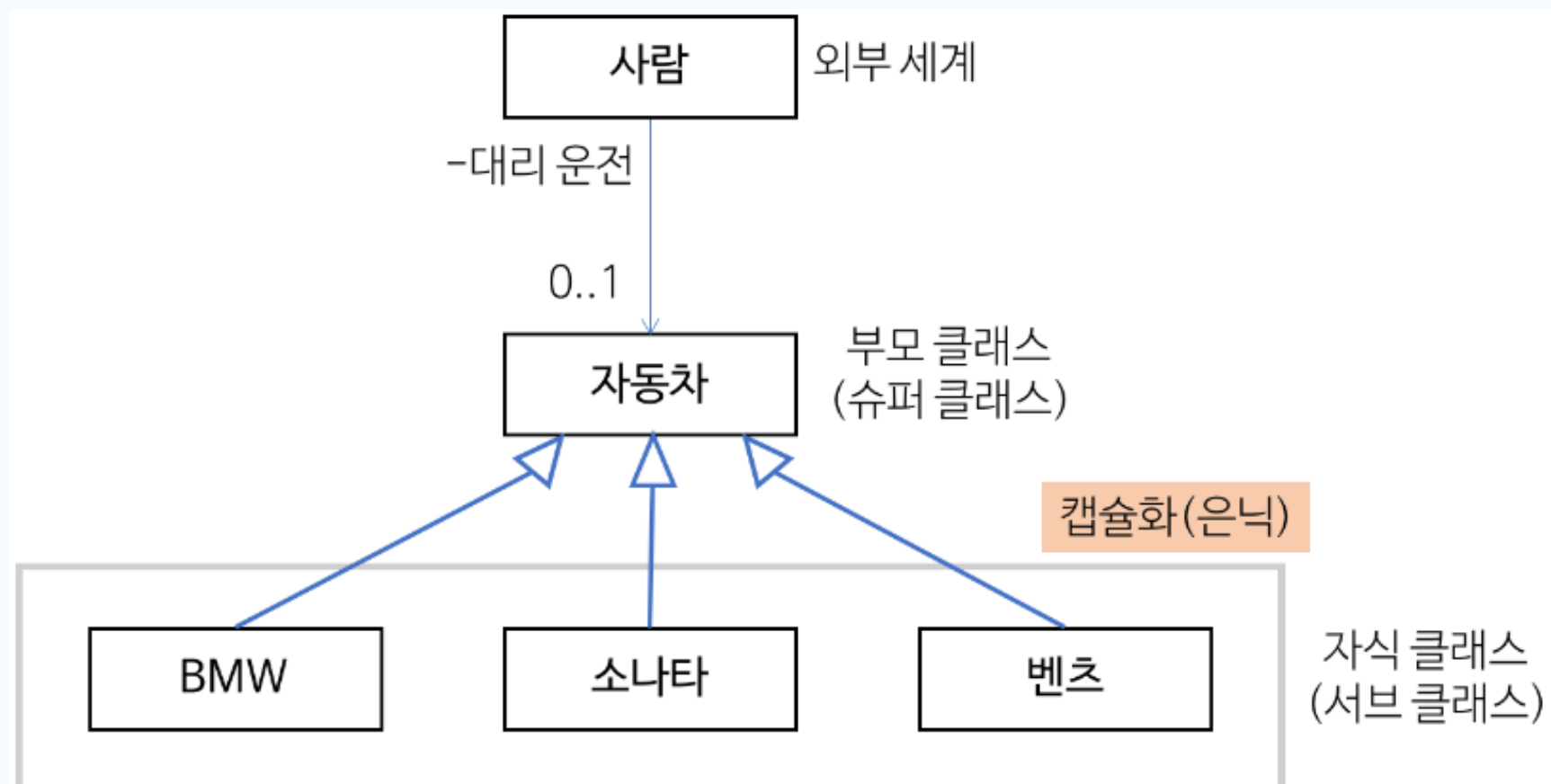

01

객체지향 기본원리 (캡슐화, 정보은닉, 상속성, 추상화, 객체의 다형성)

- 정보 은닉

정보 은닉
(information hiding)

- 캡슐화에서 가장 중요한 개념으로 다른 객체에 자신의 정보를 숨기고 연산만을 통하여 접근을 허용하는 것이다.
- 각 객체의 수정이 다른 객체에게 주는 고려되지 않은 영향(side effect)을 최소화하는 기술이다.
- 외부 객체가 특정 객체의 데이터와 함수를 직접 접근하여 사용하거나 변경하지 못하므로 유지보수와 소프트웨어 확장 시 오류를 최소화할 수 있다.



01

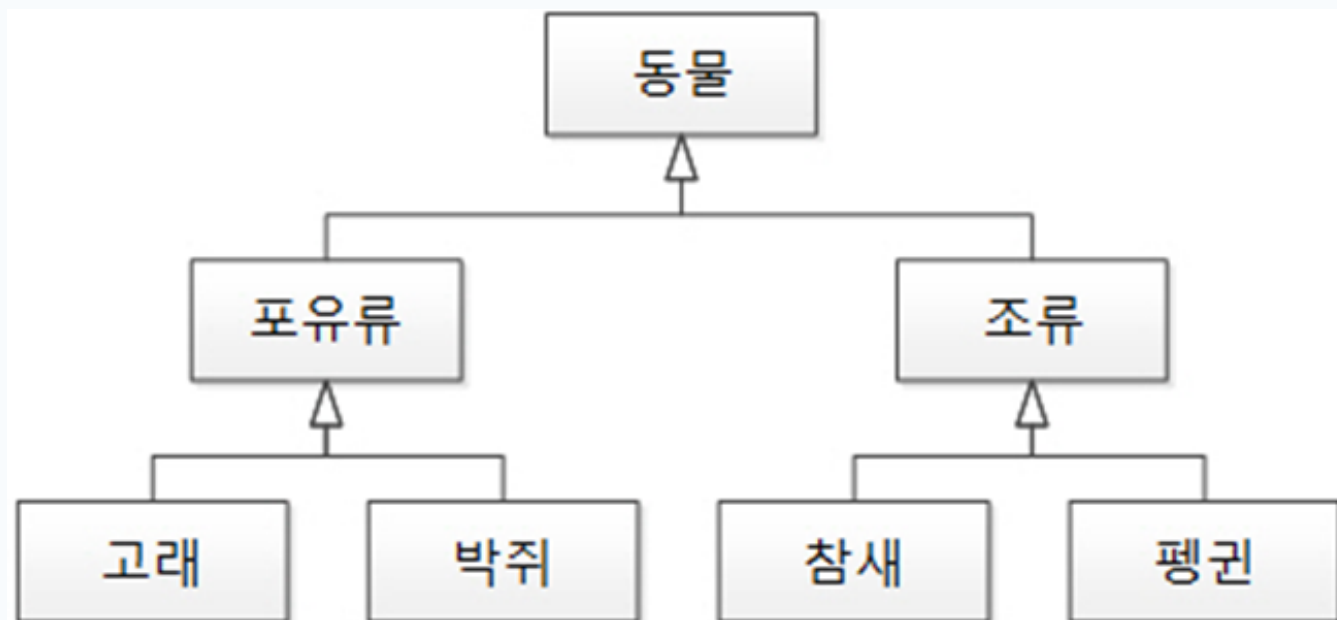
객체지향 기본원리 (캡슐화, 정보은닉, 상속성, 추상화, 객체의 다형성)

- 상속

상속 (inheritance)

- 프로그램을 쉽게 확장할 수 있도록 도와주는 수단이 된다.
- 상속은 객체 지향 패러다임에서만 구현이 가능하다.
- 상속은 정보를 공개하고 재사용할 수 있다.

- 상세화 : 부모(super) 클래스의 속성을 상속받아 자식(sub) 클래스에서 실체화 한다.
- 일반화 : 자식 클래스의 공통 특성을 추출하여 부모 클래스로 추상화 하여 정의한다.



추상화(일반화)와 구체화(상세화)

01

객체지향 기본원리 (캡슐화, 정보은닉, 상속성, 추상화, 객체의 다형성)

- 상속의 장단점과 상속의 예

장점	단점
- 재사용으로 인한 코드가 줄어든다 - 범용적인 사용이 가능하다 - 자료와 메서드의 자유로운 사용 및 추가가 가능하다	- 상위 클래스의 변경이 어려워진다 - 불필요한 클래스가 증가할 수 있다 - 상속이 잘못 사용될 수 있다

```
class Animal{
    String myClass;
    Animal() {
        myClass="동물";
    }
    void showMe() {
        System.out.println(myClass);
    }
}

class Mammalia extends Animal {
    Mammalia() {
        myClass = "포유류";
    }
}
```

```
class Whale extends Mammalia {
    Whale () {
        myClass="고래";
    }
}

public class Exam {
    public static void main(String[] args) {
        Whale whale = new Whale();
        whale.showMe();
    }
}
```


01

객체지향 기본원리 (캡슐화, 정보은닉, 상속성, 추상화, 객체의 다형성)

- 다형성

다형성 (polymorphism)

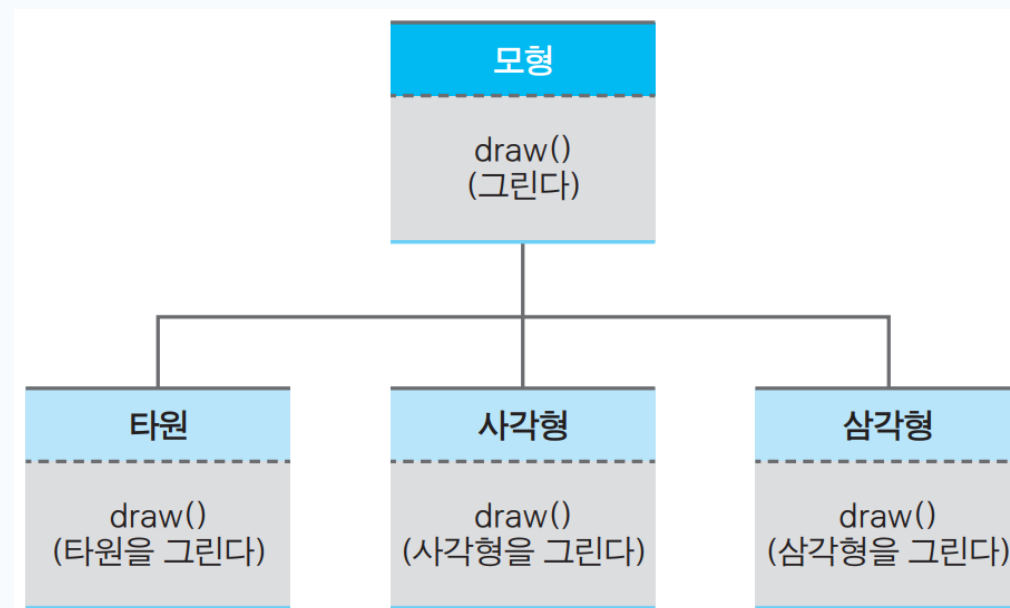
- 메시지에 의해 객체(클래스)가 연산을 수행하게 될 때 하나의 메시지에 대해 각 객체(클래스)가 가지고 있는 고유한 방법으로 응답할 수 있는 능력이다.
- '다형성'은 여러 가지 형태를 가지고 있다는 의미로 하나의 메시지에 대해 여러 가지 형태의 응답이 있다는 것을 의미한다.
- 객체(클래스)들은 동일한 메소드명을 이용하여 같은 의미의 응답을 한다.
- 응용 프로그램 상에서 하나의 함수나 연산자가 두 개 이상의 서로 다른 클래스의 인스턴스들을 같은 클래스에 속한 인스턴스처럼 수행할 수 있도록 하는 것이다.
- 많은 상이한 클래스들이 동일한 메소드명을 이용하는 능력이다.

01

객체지향 기본원리 (캡슐화, 정보은닉, 상속성, 추상화, 객체의 다형성)

- 다형성

- 다형성 예



- 모형에서 선언된 draw() 메소드는 할당되는 하위 클래스의 객체에 따라 다양한 변신을 시도하여 서로 다른 결과를 나타낸다.
 - 메시지를 요구한 메소드(draw())의 매핑을 동적으로(실행시간) 수행한다.

모형 A	상위 클래스 타입의 객체변수 A 선언
A=new 타원()	상위 클래스 타입의 객체변수 A에 타원 클래스의 객체를 생성하여 할당함
A.Draw()	타원 클래스에 기술된 draw() 메소드를 수행하여 선언된 타원을 그림
A=new 사각형 ()	상위 클래스 타입의 객체변수 A에 사각형 클래스의 객체를 생성하여 할당함
A.Draw()	사각형 클래스에 기술된 draw() 메소드를 수행하여 선언된 사각형을 그림

01

객체지향 기본원리 (캡슐화, 정보은닉, 상속성, 추상화, 객체의 다형성)

- 다형성

- 다형성 예

```
class Animal{
    String myClass;
    Animal() {
        myClass="동물";
    }
    void showMe() {
        System.out.println(myClass);
    }
}

class Mammalia extends Animal {
    Mammalia() {
        myClass = "포유류";
    }
}

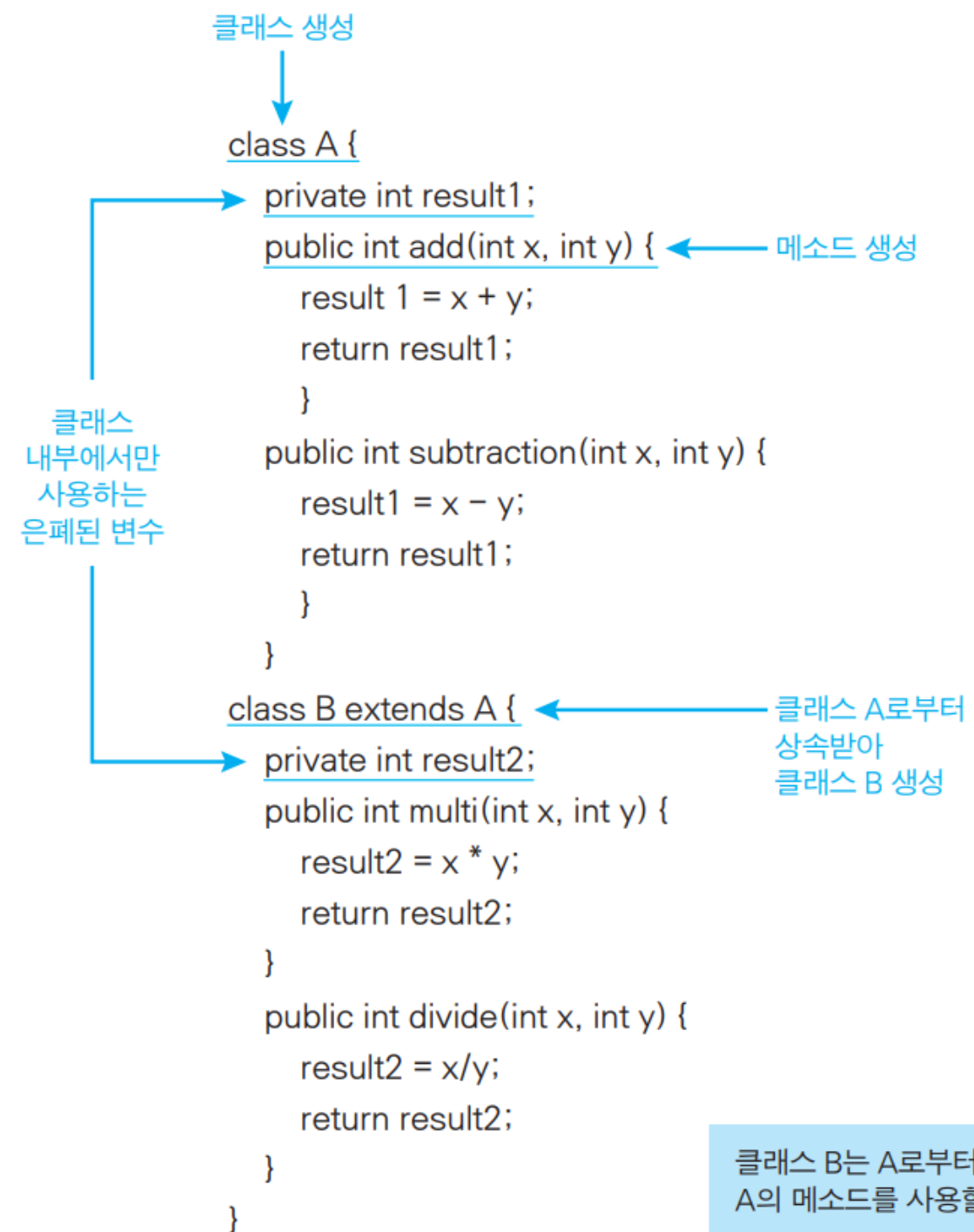
class Birds extends Animal {
    Birds() {
        myClass = "조류";
    }
}

public class Exam{
    public static void main(String[] args) {
        Animal animal;
        animal = new Birds();
        animal.showMe();
        animal = new Mammalia();
        animal.showMe();
    }
}
```

02

객체지향 언어

- JAVA



02

객체지향 언어

- JAVA

```
class TestAB
    public static void main(String args[]) {
        int temp;
        A objecta = new A();
        B objectb = new B();
        temp = objecta.add(10,20);
        System.out.println(A의 add 수행결과 + temp);
        temp = objectb.add(1,2);
        System.out.print(B의 add 수행결과 + temp);
        temp = objectg.mult(2,2);
        System.out.println(B의 multi 수행결과 + temp);

        temp = objecta.multi(20,20);
        System.out.println(A의 multi 수행결과 + temp);
    }
}
```

클래스로부터
객체의 생성

객체에게 메시지를 보냄

objecta 객체는 메소드 add()를
가지고 있지 않으므로 상위 클래스를
탐색하여 add() 메소드를 수행

objecta 객체는 multi() 메소드를
가지고 있지 않으므로 에러를 발생